MANUAL TÉCNICO Y ARQUITECTURA DE SOFTWARE

Sistema de Estimación de Peso Ganadero (BovinoWeight AI)

Especificación técnica de ingeniería, modelos de clases UML, arquitectura de inferencia en memoria sin persistencia de pesajes y gestión de usuarios operadores basada en segmentación con Detectron2, profundidad métrica con UniDepth y regresión condicional FiLM.

1. DIAGRAMA DE PROCESOS Y PIPELINE DE INFERENCIA

El flujo operativo del sistema ejecuta una secuencia estricta por etapas: primero se segmenta la silueta con Detectron2 para obtener la máscara binaria; dicha máscara pasa al clasificador de postura; seguidamente UniDepth computa el mapa métrico de profundidad; la máscara se proyecta sobre el mapa de profundidad para aislar métricamente al bovino; y finalmente, con las representaciones enmascaradas (RGB y profundidad), se direcciona al estimador o regresor correspondiente a la postura clasificada para calcular el peso exacto en kilogramos:

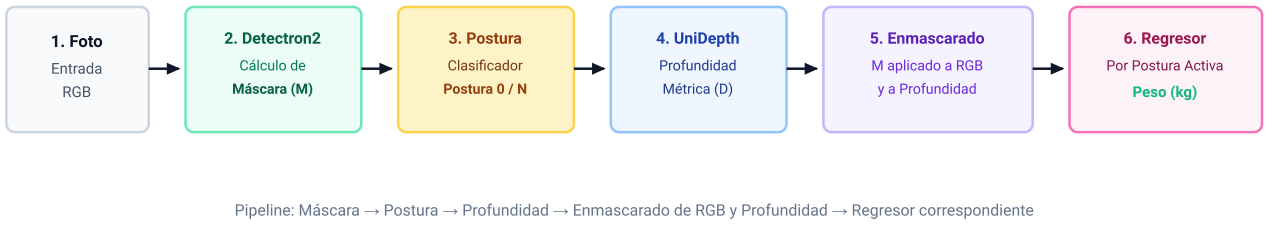


FIGURA 1: FLUJO SECUENCIAL DE INFERENCIA BIOMÉTRICA POR ETAPAS Y MODULACIÓN DE POSTURA

Etapa del Pipeline	Módulo Tecnológico	Transformación de Datos / Salida
1. Ingesta	`EstimadorView (Django)`	Carga de la imagen lateral monocular en formato RGB normalizado.
2. Máscara de Silueta	`Detectron2 (Mask R-CNN)`	Segmentación precisa del ganado bovino para aislar la silueta del entorno.
3. Clasificación de Postura	`Clasificador de Postura`	Evaluación de la pose y orientación corporal para seleccionar el regresor correspondiente.
4. Profundidad Métrica	`UniDepth`	Generación del mapa de profundidad absoluto en metros desde la óptica de cámara.

Etapa del Pipeline	Módulo Tecnológico	Transformación de Datos / Salida
5. Enmascarado Dual	`Pipeline en Memoria`	Aplicación de la máscara tanto a la imagen RGB como a la imagen de profundidad métrica.
6. Estimación de Peso	`Regresor de Postura Específico`	Cálculo ponderado del peso corporal final en kg con el modelo asociado a la postura clasificada.

2. DIAGRAMA DE CLASES UML DEL SUBSISTEMA DE INFERENCIA

El diseño orientado a objetos del backend desacopla la capa de presentación HTTP en Django de los motores de ejecución de PyTorch. Las clases de análisis se agrupan en un servicio unificado de pesaje (`PesajeEngineService`), ejecutando todo el cómputo en memoria volátil para asegurar rapidez y eliminar bloqueos transaccionales.

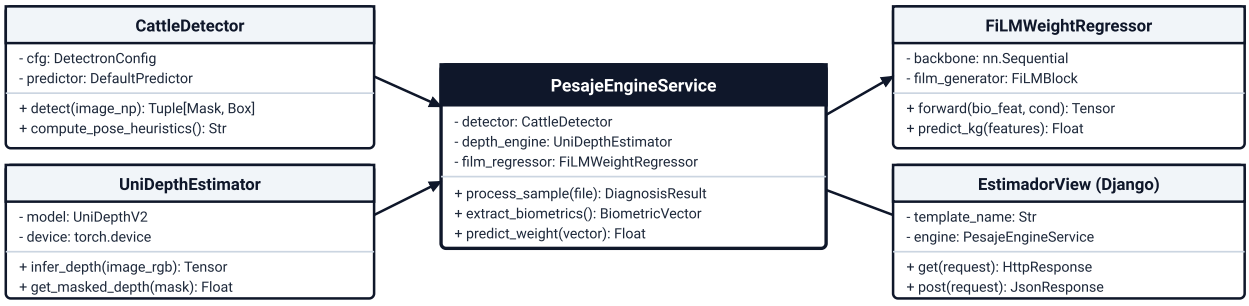


FIGURA 2: DIAGRAMA DE CLASES DE LOS MÓDULOS DE DETECCIÓN, ANÁLISIS MÉTRICO Y ORQUESTACIÓN

3. ESQUEMA DE DATOS Y ARQUITECTURA DE AUTENTICACIÓN

El sistema desacopla la persistencia de datos: la inferencia opera en memoria sin almacenamiento en tablas de pesaje, eliminando bloqueos de base de datos durante el diagnóstico. La base de datos relacional SQLite3 gestiona exclusivamente la autenticación, roles institucionales y auditoría del módulo administrativo a través de la entidad `auth_user`.

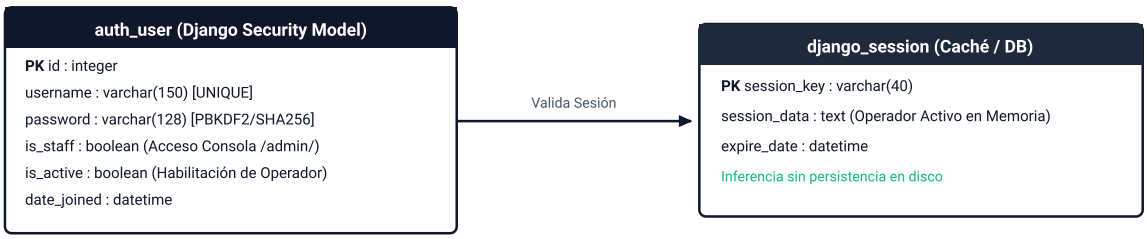


FIGURA 3: MODELO RELACIONAL DE AUTENTICACIÓN Y GESTIÓN DE SESIONES SIN PERSISTENCIA DE PESAJES

Atributo	Tipo de Dato	Restricción	Finalidad en BovinoWeight
`id`	`INTEGER`	PRIMARY KEY	Identificador único del usuario u operador del sistema.
`username`	`VARCHAR(150)`	NOT NULL, UNIQUE	Identificador de inicio de sesión (ej. `agronomo` o `computacion`).
`password`	`VARCHAR(128)`	NOT NULL	Hash criptográfico de acceso generado con algoritmo PBKDF2.
`is_staff`	`BOOLEAN`	DEFAULT FALSE	Determina acceso al panel de administración central (`/admin/`).
`is_active`	`BOOLEAN`	DEFAULT TRUE	Bandera de estado que permite suspender operadores sin eliminar sus registros de auditoría.

4. ENTORNO DE DESPLIEGUE Y CONTENEDORIZACIÓN

El sistema se distribuye en una imagen Linux contenida y reproducible construida con Docker. Debido a la carga de tensores y pesos pre-entrenados de Detectron2 y UniDepth, se requiere una gestión controlada de memoria física y caché.

> **Requisitos de Infraestructura:** Arquitectura x86_64, mínimo 8 GB de RAM libres (16 GB recomendado para evitar OOM con tensores en CPU), 50 GB de almacenamiento disponible y Docker Engine v24+.

Variable / Parámetro	Valor de Configuración	Impacto Operativo
Base Image	`python:3.11-slim-bookworm`	Entorno Debian optimizado con dependencias OpenCV y libglib.
TORCH_HUB_TRUSTED_REPOS	`lpiccinelli-eth/UniDepth`	Habilita la descarga y resolución automática del backbone ViT.
Volumen Persistente	`./data:/app/data`	Mantiene intacta la base de datos `db.sqlite3` para usuarios y sesiones entre reinicios.
Compilación Limpia	`docker compose build --no-cache`	Previene colisiones por archivos compilados `.pyc` residuales.